

VERİ

Veri Madenciliği ve Mak. Öğr. Alg. - Hafta 3



Veri Madenciliği ve Makine Öğrenmesine Giriş

Bu ders, Yapay Zekâ (YZ) ve Makine Öğrenmesi (MÖ) algoritmalarının temelini oluşturan 'veri' kavramına odaklanmaktadır.

Veri bilimi disiplini, verilerin toplanmasından analizine ve MÖ modelleri oluşturulmasına kadar geçen süreci kapsar.

Bu bölüm, veri setlerinin yapısını, farklı veri türlerini ve Python'daki bilimsel hesaplama temeli olan NumPy kütüphanesini inceleyecektir.



Yapay Zekâda Verinin Merkezi Rolü

Yapay zekâ, verileri sistematik bir şekilde analiz eder ve onlardan öğrenmek için kullanır.

Veriler, yapay zekâ modellerinin temel bileşenidir ve modelin başarısını doğrudan etkiler.

Verinin kalitesi, miktarı ve çeşitliliği, bir yapay zekâ sisteminin genelleme yeteneğini ve performansını belirler.

Veri olmadan öğrenme, karar verme veya tahmin yapma süreci gerçekleşemez.



Veri Seti (Dataset) Kavramı

Veri seti, temel olarak verilerin belirli bir sırayla düzenlendiği veri topluluğu dosyasıdır.

Bu topluluk, makine öğrenmesi algoritmalarının eğitilmesi, test edilmesi ve doğrulanması için kullanılır.

Veri setleri, genellikle satırlar (gözlemler/kayıtlar) ve sütunlardan (özellikler/değişkenler) oluşan iki boyutlu bir tablo yapısına sahiptir.

Yaygın Veri Seti Formatları

Veri setleri genel olarak CSV (Comma Separated Values) veya elektronik tablo formatında saklanır.

CSV: Metin tabanlıdır, her bir değerin virgül veya başka bir ayırıcı ile ayrıldığı düz bir dosyadır. Evrensel olarak uyumludur.

Elektronik Tablo (Excel/Sheets): Daha zengin formatlama seçenekleri sunar ancak yazılımlar arası uyumluluk bazen sorun yaratabilir.

Modern Veri Ambarı Formatları: Parquet, ORC ve HDF5 gibi yüksek performanslı, sıkıştırılmış formatlar büyük veri setleri için tercih edilir.

Açık Veri Kaynaklarının Önemi

Açık veri kaynakları, araştırmacılara ve öğrencilere model geliştirmek için zengin veri setleri sunar.

Kaggle: Veri bilimi yarışmaları ve büyük bir topluluğa sahip platformdur. Çok çeşitli kategorilerde veri setleri barındırır.

UCI Makine Öğrenmesi Deposu: Akademik çalışmalar için standartlaşmış, uzun süredir kullanılan veri setlerinin bulunduğu bir havuzdur.

Diğer Kaynaklar: Devlet kurumlarının (TÜİK, CDC) yayınladığı kamu verileri ve akademik kurumların özel arşivleri de önemlidir.



Veri Kalitesinin Kritikliği

Veri kalitesi, modelin performansını ve güvenilirliğini belirler.

Eksik Veri: Bazı gözlemlerdeki değerlerin bulunmaması. Bu durum, doldurma (imputation) veya silme gerektirebilir.

Gürültülü Veri: Yanlış, hatalı veya bozuk kayıtlar içeren veridir. Özellikle sensör verilerinde veya manuel girişte yaygındır.

Tutarsız Veri: Aynı özelliğin farklı formatlarda veya çelişkili değerlerle kaydedilmesi (Örn: 'US' ve 'United States' olarak kaydedilen ülke bilgisi).



Veri Ön İşleme (Preprocessing) Aşaması

Veri ön işleme, ham veriyi makine öğrenmesi algoritmalarının anlayabileceği ve işleyebileceği formata dönüştürme sürecidir.

Temizleme (Cleaning): Eksik değerleri giderme, gürültüyü azaltma ve tutarsızlıkları düzeltme.

Dönüştürme (Transformation): Veriyi belirli bir aralığa ölçekleme (scaling) veya normalleştirme (normalization).

Özellik Mühendisliği (Feature Engineering): Model performansını artırmak için mevcut verilerden yeni özellikler türetme.



Veri Türlerine Genel Bakış

Veri türleri, üzerinde hangi matematiksel ve istatistiksel işlemlerin uygulanabileceğini belirler.

Ana gruplar şunlardır:

Sayısal (Quantitative) Veri: Matematiksel anlamı olan, ölçülebilir değerler.

Kategorik (Qualitative) Veri: Sınıflara veya gruplara ayırmak için kullanılan değerler.

Sıralı (Ordinal) Veri: Kategorik olmakla birlikte doğal bir sıraya sahip olan veriler.



Sayısal Veri (Numerical Data)

Sayısal veri, ölçülebilen ve aritmetik işlemlerin anlamlı olduğu değişkenlerdir. Örnekler: Kişilerin yaşları, ev fiyatları, sıcaklık ölçümleri, maaşlar. Bu veri türü, ortalama, medyan, standart sapma gibi istatistiksel analizlere uygundur. Sayısal veriler genellikle iki ana alt türe ayrılır: Ayırık ve Sürekli.

Sayısal Veri Alt Türü: Ayırık (Discrete) Veri

Ayrık veriler, sayılabilir bir dizi içinde yalnızca belirli tam sayı değerlerini alabilir. Bu değerler arasında sonsuz sayıda ara değer bulunmaz. Örnekler: Bir evdeki oda sayısı (1, 2, 3...), bir sınıftaki öğrenci sayısı, atılan bir zarın sonucu. Genellikle sayma işlemlerinden elde edilir.

Sayısal Veri Alt Türü: Sürekli (Continuous) Veri

Sürekli veriler, belirli bir aralık içinde sonsuz sayıda değer alabilir.
Örnekler: Bir kişinin boyu (175.5 cm, 175.51 cm...), sıcaklık (25.3°C), zaman, ağırlık.
Bu veriler genellikle ölçüm cihazları ile elde edilir ve hassasiyet, ölçüm aletinin limitleriyle sınırlıdır.

Kategorik Veri (Categorical Data)

Kategorik veriler, gözlemleri gruplara veya kategorilere ayırmak için kullanılır. Bu verilerin değerleri matematiksel anlam taşımaz, ancak frekansları hesaplanabilir.

Örnekler: Cinsiyet (Kadın/Erkek), renkler (Mavi, Kırmızı), evet/hayır cevapları, şehirler.

Kategorik veriler genellikle metin veya ikili kodlar (0/1) şeklinde depolanır.

Kategorik Veri Alt Türü: Nominal Veri

Nominal veriler, kategoriler arasında doğal bir sıralama veya hiyerarşi bulunmayan türlerdir.

Tek fark, isimleridir (nominal = isimsel).

Örnekler: Göz rengi, medeni durum, tutulan takım.

Üzerlerinde yapılabilecek tek anlamlı istatistiksel işlem mod (en sık görülen değer) ve frekans sayımıdır.



Kategorik Veri Alt Türü: İkili (Binary/Dichotomous) Veri

İkili veri, yalnızca iki olası kategoriye sahip olan nominal verinin özel bir durumudur.

Bu iki durum genellikle 0 ve 1, Evet ve Hayır, Başarılı ve Başarısız olarak kodlanır. Örnekler: Kredi notu alındı mı (Evet/Hayır), bir ürünün kusurlu olup olmadığı (0/1).



Sıralı Veri (Ordinal Data)

Sıralı veriler, kategorik olmakla birlikte, bu kategoriler arasında doğal bir sıralama veya düzen barındırır.

Ancak, kategoriler arasındaki farklar (aralıklar) eşit veya ölçülebilir değildir. Örnekler: Eğitim dereceleri (İlk/Orta/Lise/Üniversite), müşteri memnuniyeti derecelendirmesi (Zayıf/Orta/İyi/Pekiyi).



Sıralı Verideki Kısıtlamalar

Sıralı verinin temel kısıtlaması, kategoriler arasındaki farkların sayısal olarak anlamlı olmamasıdır.

Örnek: 'İyi' ve 'Pekiyi' arasındaki fark, 'Zayıf' ve 'Orta' arasındaki farkla aynı büyüklükte olmayabilir.

Bu nedenle, sıralı verilere doğrudan aritmetik ortalama uygulamak genellikle yanıltıcıdır.



Veri Ölçüm Ölçekleri (Measurement Scales)

İstatistik ve veri biliminde verilerin nasıl ölçüldüğünü ve hangi analizlerin uygun olduğunu belirleyen dört temel ölçek vardır:

1. Nominal (Adlandırma)
2. Ordinal (Sıralama)
3. Interval (Aralık)
4. Ratio (Oran)



Ölçüm Ölçeği: Aralık (Interval) Ölçeği

Aralık ölçeği, sıralama ve eşit aralık özelliklerini taşır.

İki değer arasındaki farklar anlamlıdır (Örn: 20°C ile 30°C arasındaki fark 10°C 'dir).

Ancak, doğal bir mutlak sıfır noktası yoktur. Sıfır, bir yokluk anlamına gelmez (Örn: 0°C , ısıнын yokluğu anlamına gelmez).

Örnekler: Sıcaklık (Celsius veya Fahrenheit), takvim tarihleri.

Ölçüm Ölçeği: Oran (Ratio) Ölçeği

Oran ölçeği, en bilgilendirici ölçek türüdür. Nominal, sıralı, eşit aralık ve ek olarak mutlak sıfır noktası özelliklerine sahiptir.

Mutlak sıfır, özelliğin yokluğunu temsil eder (Örn: 0 kg ağırlık, ağırlığın olmaması demektir).

Çarpma ve bölme dahil tüm aritmetik işlemler anlamlıdır.

Örnekler: Yaş, boy, ağırlık, gelir, ev fiyatları.

Kategorik Verinin Sayısallaştırılması: Kodlama (Encoding)

Makine öğrenmesi algoritmalarının çoğu yalnızca sayısal girdi beklediği için, kategorik veriler kodlanmalıdır.

Hatırlatma: Nominal ve Sıralı veriler farklı şekillerde kodlanır.

Yanlış kodlama, modelin yanlış varsayımlar öğrenmesine neden olabilir (Örn: Nominal veriye rastgele sayısal sıra vermek).



Kodlama Yöntemi: One-Hot Encoding

One-Hot Encoding (OHE), nominal kategorik veriler için kullanılır. Her benzersiz kategori, veri setinde yeni bir sütun olarak temsil edilir. Eğer bir gözlem o kategoriye aitse 1, değilse 0 değeri atanır. Avantajı: Algoritmanın kategoriler arasında rastgele bir sıralama öğrenmesini engeller. Dezavantajı: Çok fazla kategori varsa ('Curse of Dimensionality'), veri setinin boyutunu aşırı derecede artırabilir.

Kodlama Yöntemi: Label Encoding

Label Encoding, her bir kategoriye sayısal bir değer atar (Örn: Zayıf=1, Orta=2, İyi=3, Pekiyi=4).

Bu yöntem, yalnızca verinin doğal bir sıralaması (ordinalite) varsa uygundur. Eğer nominal veriye uygulanırsa, algoritma 4'ün 1'den 'daha iyi' veya 'daha büyük' olduğunu varsayabilir, bu da model hatasına yol açar.

NumPy Kütüphanesine Giriş

NumPy (Numerical Python), açık kaynak kodlu bilimsel hesaplamalar için kullanılan bir Python kütüphanesidir.

Python'da büyük, çok boyutlu diziler ve matrislerle yüksek performanslı işlemler yapmak için standarttır.

NumPy, Pandas, SciPy, Matplotlib ve Scikit-learn gibi diğer tüm popüler veri bilimi ve makine öğrenmesi kütüphanelerinin temelini oluşturur.



NumPy'in Gücü: Performans ve Hız

Geleneksel Python listeleri ile yapılan döngüsel işlemler yavaştır. NumPy, C ve Fortran dillerinde optimize edilmiş arka uca sahiptir ve işlemleri vektörleştirilmiş (vectorized) biçimde gerçekleştirir. Vektörleştirme, döngüleri Python yorumlayıcısından çıkarıp düşük seviyeli, optimize edilmiş kodda çalıştırmak anlamına gelir. Bu, özellikle milyonlarca veri noktasını içeren büyük veri setleri için performansta dramatik artış sağlar.

NumPy'in Temel Nesnesi: ndarray

NumPy kütüphanesinin temelinde, 'N-Dimensional Array' (N-Boyutlu Dizi) anlamına gelen ndarray nesnesi bulunur.

Bu diziler, standart Python listelerinden daha hızlı ve işlevseldir.

ndarray nesneleri, aynı veri tipine sahip öğeleri içerir (homojen yapı).

Bu homojenlik, bellekte daha verimli depolama ve CPU'nun SIMD (Single Instruction, Multiple Data) komut setlerini kullanarak hızlı erişim sağlar.

ndarray Özellikleri: Boyut ve Şekil

Bir ndarray'in temel özellikleri şunlardır:

ndim (Boyut sayısı): Dizinin eksen sayısını belirtir (1D, 2D, 3D vb.). Bir matrisin 2 boyutu vardır.

shape (Şekil): Dizinin her boyuttaki eleman sayısını gösteren bir tuple'dır (Örn: 3x4'lük bir matrisin şekli (3, 4) olur).

size (Boyut): Dizideki toplam eleman sayısını verir (Örn: 3x4'lük matrisin boyutu 12'dir).

ndarray Özellikleri: Veri Tipleri (dtype)

ndarray'in tüm elemanları aynı veri tipine sahip olmak zorundadır (Örn: int32, float64).

Bu 'dtype' (data type) özelliği, bellekte ne kadar yer kaplanacağını ve hangi işlemlerin uygulanabileceğini belirler.

Python'daki listeler heterojen (farklı tipleri barındırabilir) iken, NumPy dizileri homojen olmak zorundadır, bu da hız avantajı sağlar.



ndarray Oluşturma Yöntemleri (1)

En basit yöntem, mevcut bir Python listesini veya iç içe listeyi (matris için) 'np.array()' fonksiyonu ile dönüştürmektir.

Tek Boyutlu Dizi (Vektör): np.array()

İki Boyutlu Dizi (Matris): np.array([,])

NumPy, varsayılan olarak veri tipini (dtype) otomatik olarak çıkarır, ancak bu, 'dtype' parametresi ile manuel olarak da ayarlanabilir.

ndarray Oluşturma Yöntemleri (2)

Büyük veri setleri veya önceden tanımlanmış boyutlar için yer tutucu fonksiyonlar kullanılır:

`np.zeros((shape))`: Belirtilen şekilde tamamen sıfırlardan oluşan bir dizi oluşturur.

`np.ones((shape))`: Tamamen birlerden oluşan bir dizi oluşturur.

`np.empty((shape))`: Bellekte ayrılmış, ancak rastgele (önceden kalan) değerler içeren bir dizi oluşturur (en hızlı yöntemdir).

ndarray Oluşturma Yöntemleri (3)

`np.arange(start, stop, step)`: Belirtilen aralıkta ve adımda değerler içeren bir dizi oluşturur (Python'daki `range()` fonksiyonunun dizi versiyonu).

`np.linspace(start, stop, num)`: Belirtilen başlangıç ve bitiş noktaları arasında 'num' kadar eşit aralıklı nokta oluşturur. Makine öğrenmesi algoritmaları ve çizim işlemleri için hayati öneme sahiptir.



Dizi İşlemleri: İndeksleme (Indexing)

NumPy dizilerinde indeksleme, Python listelerindekine benzer şekilde sıfırdan başlar.

Tek boyutlu dizilerde: $a[i]$

Çok boyutlu dizilerde: Her boyut için bir indeks belirtilir (Örn: $a[\text{satır}, \text{sütun}]$ veya $a[\text{eksen0}, \text{eksen1}, \dots]$).

Negatif indeksleme (sondan başa doğru sayma) de desteklenir.

Dizi İşlemleri: Dilimleme (Slicing)

Dilimleme, büyük bir diziden bir alt dizi (sub-array) çıkarmak için kullanılır: `a[start:stop:step]`.

İki boyutlu dizilerde dilimleme, hem satırlar hem de sütunlar için ayrı ayrı yapılabilir (Örn: `a[0:2, 1:]`).

NumPy'da dilimleme ile oluşturulan alt diziler, orijinal dizinin bellekteki verisine referans verir (View), kopyasını oluşturmaz. Bu, bellek verimliliği sağlar.

Boolean (Maskeleye) indeksleme

Boolean indeksleme, dizideki değerlere bir koşul uygulayarak 'True' olan elemanları seçmek için kullanılır.

Örn: `dizi[dizi > 5]` ifadesi, dizideki 5'ten büyük tüm elemanları döndürür.

Bu güçlü teknik, veri filtreleme, anormallik tespiti ve eksik veri yönetimi gibi veri madenciliği görevlerinde yaygın olarak kullanılır.



Temel Aritmetik İşlemler

NumPy, diziler üzerinde doğrudan eleman bazında (element-wise) aritmetik işlemleri destekler.

Toplama, çıkarma, çarpma ve bölme gibi işlemler, Python listelerinin aksine, dizideki her elemana ayrı ayrı uygulanır.

Bu işlemler, Python döngülerine kıyasla çok daha hızlıdır.

Örn: $A + B$, A dizisinin her elemanını B dizisinin karşılık gelen elemanı ile toplar.

Matematiksel Fonksiyonlar (Ufuncs)

Ufuncs (Evrensel Fonksiyonlar), ndarray'in her elemanına hızlı bir şekilde matematiksel fonksiyonlar uygulamak için tasarlanmıştır.

Örnekler: `np.sin()`, `np.cos()`, `np.sqrt()`, `np.log()`, `np.exp()`.

Bu fonksiyonlar, döngü yazma ihtiyacını ortadan kaldırır ve performansı optimize eder.

Ufuncs, sadece tek bir diziyi girdi olarak alıp sonuç dizisini döndürebilir (tekli ufunc) veya iki diziyi girdi olarak alabilir (ikili ufunc).



Toplama (Aggregation) Fonksiyonları

NumPy, dizinin tamamını veya belirli eksenlerini özetleyen istatistiksel fonksiyonlar sunar.

`np.sum()`: Dizideki tüm elemanların toplamını hesaplar.

`np.mean()`: Ortalama değeri hesaplar.

`np.std()`: Standart sapmayı hesaplar.

`np.min()`, `np.max()`: Minimum ve maksimum değerleri bulur.

Bu fonksiyonlar 'axis' parametresi kullanılarak belirli bir eksen (satır veya sütun) boyunca çalıştırılabilir.



Eksen Boyunca Toplama (Axis Aggregation)

Çok boyutlu bir dizide, 'axis' parametresi, işlemin hangi boyut boyunca yapılacağını belirtir.

'axis=0' genellikle sütunlar boyunca (her sütunun toplamı/ortalaması) işlem yapılmasını ifade eder.

'axis=1' genellikle satırlar boyunca (her satırın toplamı/ortalaması) işlem yapılmasını ifade eder.

Bu, matris işlemlerinde ve makine öğrenmesi algoritmalarında (Örn: Normalizasyon) kritik öneme sahiptir.



Broadcasting Kavramı

Broadcasting, NumPy'in farklı şekillere sahip diziler üzerinde aritmetik işlemleri mümkün kılan bir mekanizmadır.

NumPy, daha küçük olan diziye, işlem yapılabilmesi için daha büyük olan diziye uyacak şekilde 'esnetir' veya 'yayınlar' (broadcast eder).

Temel kural: İşleme giren dizilerin boyutları, sondan başlayarak karşılaştırıldığında ya eşit olmalı ya da biri 1 olmalıdır.



Broadcasting Uygulaması

En basit broadcasting örneği, bir skaler (tek bir sayı) ile bir vektör veya matrisi toplamaktır.

Örn: Bir matrise 5 eklemek, 5'in matristeki her elemana otomatik olarak eklenmesini sağlar.

Bu, tüm veri setini belirli bir sabit değerle ölçeklemek veya kaydırmak gerektiğinde performanstan ödün vermeden hızlı işlem yapmayı sağlar.



Yeniden Şekillendirme (Reshaping) İşlemleri

'reshape()' fonksiyonu, dizinin boyutunu (shape) değiştirerek orijinal veri yapısını korurken, verinin yeni bir görünümünü (view) oluşturur.

Önemli kural: Yeniden şekillendirilen dizinin toplam eleman sayısı (size), orijinal dizinin eleman sayısı ile aynı kalmalıdır.

Bu, Makine Öğrenmesinde (özellikle derin öğrenmede) veriyi modele uygun formata (Örn: (N_örnek, N_özellik) veya (N_örnek, yükseklik, genişlik, kanal)) sokmak için hayati bir işlemdir.



Yeni Boyut Ekleme ve Transpozisyon

`np.newaxis`: Bir diziye yeni bir boyut (eksen) eklemek için kullanılır. Özellikle tek boyutlu vektörlerin matris işlemlerine uygun hale getirilmesi gerektiğinde önemlidir.

'T' (Transpose): Bir matrisin satırlarını sütun, sütunlarını ise satır yaparak transpozisini almak için kullanılır.

Transpozisyon, Lineer Cebirde ve özellikle dot product (nokta çarpımı) işlemlerinde standart bir gerekliliktir.

Lineer Cebir Temelleri ve NumPy

NumPy'in alt modülü olan 'numpy.linalg', lineer cebir işlemleri için gerekli araçları sağlar.

Matris Çarpımı: '@' operatörü veya 'np.dot()' fonksiyonu kullanılarak matrislerin çarpımı gerçekleştirilir.

'np.linalg.inv()': Bir matrisin tersini (inverse) hesaplar.

'np.linalg.det()': Bir matrisin determinantını hesaplar.

Makine öğrenmesi modellerinde (Örn: Regresyon) matris işlemleri yoğun olarak kullanılır.



NumPy ve Bellek Verimliliği

NumPy dizileri, bellekte ardışık (contiguous) bloklar halinde depolanır. Bu ardışık depolama, CPU'nun önbelleğini (cache) daha verimli kullanmasını sağlar ve hızlı veri erişimine olanak tanır. Python listeleri ise, belleğin farklı yerlerine dağılmış nesnelere referanslar tutar, bu da erişim maliyetini artırır. Bu fiziksel depolama farkı, NumPy'ın neden büyük veri setlerinde Python listelerine göre kat kat daha hızlı olduğunu açıklar.

NumPy ve İstatistiksel Dağılımlar

'numpy.random' modülü, çeşitli olasılık dağılımlarından rastgele sayılar üretmek için kullanılır.

`np.random.rand()`: 0 ile 1 arasında tekdüze (uniform) dağılmış rastgele sayılar üretir.

`np.random.randn()`: Standart normal dağılımdan (ortalama 0, standart sapma 1) rastgele sayılar üretir.

`np.random.randint()`: Belirtilen aralıkta rastgele tam sayılar üretir.

Bu işlevler, simülasyonlar, model başlatma (initialization) ve sentetik veri üretimi için önemlidir.

Veri Biliminde ndarray Kullanımı (Örnek)

Ölçekleme (Scaling), veri ön işleme aşamasının kritik bir parçasıdır. Standardizasyon (Z-Score): Veriyi ortalaması 0, standart sapması 1 olacak şekilde dönüştürme. NumPy ile kolayca yapılabilir: $(X - \text{np.mean}(X, \text{axis}=0)) / \text{np.std}(X, \text{axis}=0)$ Normalizasyon (Min-Max): Veriyi 0 ile 1 arasına sıkıştırma. Genellikle görüntü işleme ve sinir ağlarında kullanılır.

Veri Seti Kavramlarının Tekrarı

Veri, YZ modellerinin temelidir ve doğru veri türünün anlaşılması, doğru model seçimini sağlar.

Sayısal Veri (Numerical): Ölçülebilir, aritmetik işlemler anlamlıdır (Yaş, Fiyat).
Ayrık veya Sürekli olabilir.

Kategorik Veri (Categorical): Gruplama için kullanılır (Cinsiyet, Renkler).

Sıralı Veri (Ordinal): Kategoriler arasında bir sıra vardır, ancak aralıklar eşit değildir (Memnuniyet seviyesi).



Veri Toplama ve Çıkarım (Inference)

Veri Toplama: Veri setinin oluşturulması aşamasıdır. Güvenilir ve tarafsız kaynaklardan veri elde etmek önemlidir.

Veri Çıkarım (Inference): Eğitilmiş bir yapay zekâ modelinin, daha önce görmediği yeni veriler üzerinde tahminler veya kararlar üretmesi sürecidir. Veri setinin kalitesi, çıkarımın güvenilirliğini doğrudan etkiler.



Veri Önyargısı (Data Bias) ve Etkisi

Veri önyargısı, veri setinin gerçek dünyadaki dağılımı veya popülasyonu doğru şekilde temsil etmemesidir.

Önyargılı veriyle eğitilen modeller, önyargılı veya ayrımcı sonuçlar üretir (Örn: Belirli bir demografik grubu yanlış sınıflandırma).

Örnekler: Tarihsel önyargı, temsiliyet önyargısı, ölçüm önyargısı.

Veri setlerinin dikkatli bir şekilde denetlenmesi ve dengelenmesi, YZ etiği açısından kritik öneme sahiptir.



Model Değerlendirme Metrikleri

Modelin performansı, ayrılmış test veri setleri kullanılarak değerlendirilir.

Sınıflandırma Metrikleri: Doğruluk (Accuracy), Kesinlik (Precision), Geri Çağırma (Recall), F1 Skoru.

Regresyon Metrikleri: Ortalama Mutlak Hata (MAE), Ortalama Karekök Hata (RMSE).

Bu metrikler, modelin ne kadar iyi öğrendiğini ve genelleme yapabildiğini gösterir.

Büyük Veri (Big Data) Bağlamında NumPy

NumPy, tek bir makinenin belleğine sığabilen (in-memory) büyük veri setleri için idealdir.

Ancak petabayt düzeyindeki veriler için Dask veya Spark gibi dağıtılmış bilgi işlem araçları kullanılır.

Bu araçlar dahi, arka planda hesaplama çekirdeği olarak genellikle NumPy'in ndarray yapısını kullanır.

Veri bilimciler, büyük veriyi işlerken NumPy'in performansı nedeniyle parçalara ayırma (chunking) yöntemini tercih ederler.



ndarray Kopyalama vs. Görünüm (View)

NumPy'da iki tür kopya işlemi vardır:

1. Derin Kopya (Deep Copy - 'copy()'): Verinin tam bir kopyasını oluşturur, bellekte yeni bir yer tahsis eder. Orijinal dizideki değişiklikler kopyayı etkilemez.
2. Görünüm (View - Dilimleme): Yeni bir dizi objesi oluşturur ancak veriyi paylaşır. Görünümde yapılan değişiklikler orijinal diziyi etkiler ve tam tersi.

Veri Kümeleme ve Birleştirme

NumPy, birden fazla diziyi tek bir dizide birleştirmek için fonksiyonlar sunar:

'np.concatenate()': Dizileri mevcut bir eksen boyunca birleştirir.

'np.vstack()': Dizileri dikey olarak (satır bazında) yığınlar.

'np.hstack()': Dizileri yatay olarak (sütun bazında) yığınlar.

Bu işlevler, farklı veri kaynaklarından gelen veya farklı özellik kümeleri içeren verileri tek bir eğitim matrisinde toplamak için gereklidir.

NumPy Dizilerinde Karşılaştırma İşlemleri

Karşılaştırma operatörleri ($>$, $<$, $==$, $!=$, $>=$, $<=$) ndarray'ler üzerinde eleman bazında çalışır ve boolean (doğru/yanlış) değerlerden oluşan bir dizi döndürür.

Bu boolean dizisi, koşullu indeksleme (maskeleme) için kullanılır.

Örn: $A == 5$, A dizisindeki hangi elemanların 5'e eşit olduğunu gösterir.

Mantıksal Operasyonlar

`np.any()`: Bir boolean dizisindeki en az bir elemanın True olup olmadığını kontrol eder.

`np.all()`: Bir boolean dizisindeki tüm elemanların True olup olmadığını kontrol eder.

Bu fonksiyonlar, bir dizi veya matrisin belirli bir kriteri tamamen karşılayıp karşılamadığını hızlıca anlamak için kullanılır.

NumPy'da Dosya I/O İşlemleri

NumPy, dizileri diskte hızlı ve verimli bir şekilde kaydetmek ve yüklemek için özel ikili formatlar sunar.

'np.save(filename, array)': Diziyi ikili '.npy' formatında kaydeder (tek dizi).

'np.load(filename)': Kaydedilmiş '.npy' dosyasını yükler.

'np.savez(filename, array1, array2)': Birden fazla diziyi tek bir sıkıştırılmış '.npz' dosyasında kaydeder.

Bu, büyük dizileri Python'ın pickle mekanizmasından çok daha hızlı bir şekilde saklamayı sağlar.



NumPy ve Pandas İlişkisi

Pandas kütüphanesi (DataFrame nesnesi), veri biliminde yaygın olarak kullanılan etiketli (labeled) veri yapıları sağlar.

Pandas, veriyi analiz etme ve manipülasyon için yüksek seviyeli araçlar sunar. Pandas DataFrame'lerinin çekirdeğinde, verinin depolandığı yer olarak hala NumPy ndarray'leri bulunur.

NumPy, düşük seviyeli, yüksek performanslı matematiksel işlemleri sağlarken, Pandas veri yönetimi ve temizliği için kullanılır.

Eksik Veri Temsili: NaN

NumPy, bilimsel hesaplamalarda eksik veya tanımsız değerleri temsil etmek için IEEE 754 standardına uygun 'Not a Number' (NaN) değerini kullanır. NaN değerleri yalnızca float (kayan nokta) tiplerinde temsil edilebilir. 'np.isnan()' fonksiyonu, dizideki hangi elemanların NaN olduğunu kontrol eden bir boolean maske döndürür. Veri ön işlemede, NaN'ları tespit etmek ve onları ortalama veya medyan gibi değerlerle doldurmak yaygın bir uygulamadır.

Gelişmiş ndarray İşlemleri: Tekrarlama

'np.repeat()': Dizinin her bir elemanını n kez tekrarlayarak yeni bir dizi oluşturur.
'np.tile()': Dizinin tamamını n kez tekrarlayarak (fayanslama/döşeme) yeni bir dizi oluşturur.

Bu fonksiyonlar, özellikle veri setlerini sentetik olarak büyütmek veya belirli bir yapıyı bir makine öğrenmesi modeline girdi olarak sunmak için kullanılır.



Dizi Bölme (Splitting) İşlemleri

NumPy, dizileri birden fazla parçaya ayırmak için fonksiyonlar sunar:

'np.split()': Diziyi belirli indekslere göre eşit veya belirtilen parçalara ayırır.

'np.hsplit()': Diziyi yatay olarak (sütunlar boyunca) böler.

'np.vsplit()': Diziyi dikey olarak (satırlar boyunca) böler.

Bu, eğitim, doğrulama ve test kümelerini ayırmak (train-test split) gibi veri bilimi görevlerinde temeldir.



Fonksiyonel Programlama ve NumPy

NumPy, dizilere özel fonksiyonları uygulamak için doğrudan araçlar sunar. Bu genellikle ufuncs ile yapılır, ancak karmaşık özel mantık gerektiren durumlarda '`np.vectorize()`' kullanılabilir. '`np.vectorize()`', aslında döngüsel bir işlemi kapsar, ancak kullanımı daha temizdir ve Python fonksiyonlarının `ndarray` üzerinde eleman bazında çalıştırılmasını sağlar.

NumPy ve Görüntü İşleme (Image Processing)

Bir dijital görüntü, genellikle 3 boyutlu bir ndarray olarak temsil edilir:

Boyut 1: Yükseklik (Satırlar)

Boyut 2: Genişlik (Sütunlar)

Boyut 3: Renk Kanalları (RGB = 3 katman)

NumPy'in hızlı matris operasyonları, görüntü filtreleme, yeniden boyutlandırma ve maskeleme gibi temel görüntü işleme görevlerini (Örn: Sobel filtreleri) son derece verimli hale getirir.



NumPy ve Bellek Haritalama (Memory Mapping)

Bellek haritalama (Memory Mapping), bir dosyanın içeriğini doğrudan bellekteki bir ndarray'miş gibi ele almayı sağlar.

Bu, diske sığan ancak belleğe sığmayacak kadar büyük olan dosyaların tamamını yüklemekten, sadece gerektiğinde parçalarını okumaya olanak tanır.

'np.memmap' fonksiyonu bu işlevi sağlar, büyük veri setleriyle çalışan bilimsel uygulamalar için önemlidir.

Dizideki Değerleri Sınırlama (Clipping)

'np.clip(array, min_val, max_val)' fonksiyonu, dizideki değerlerin belirli bir minimum ve maksimum aralığın dışına çıkmasını engeller.
Eğer bir değer 'max_val'dan büyükse, bu değere 'max_val' atanır.
Eğer bir değer 'min_val'dan küçükse, bu değere 'min_val' atanır.
Aykırı değerlerin (outliers) etkisini sınırlamak ve belirli aralık gereksinimlerini karşılamak için kullanılır.

Dizideki Aykırı Değerler (Outliers)

Aykırı değerler, veri setindeki diğer gözlemlerden önemli ölçüde farklı olan veri noktalarıdır.

Bu değerler, özellikle ortalama gibi istatistiksel hesaplamaları ve makine öğrenmesi modellerini bozabilir.

NumPy ve istatistiksel metotlar (Örn: Z-Skoru veya Çeyreklikler Arası Aralık – IQR) kullanılarak aykırı değerler tespit edilebilir ve 'np.clip' veya silme işlemleriyle yönetilebilir.



NumPy ve Optimizasyon

NumPy, dahili olarak BLAS (Basic Linear Algebra Subprograms) ve LAPACK (Linear Algebra PACKage) gibi yüksek düzeyde optimize edilmiş kütüphaneleri kullanır.

Bu kütüphaneler genellikle çok çekirdekli işlemcilerde paralel çalışacak şekilde tasarlanmıştır.

Bu alt yapı, bilimsel Python ekosisteminin neden akademik araştırmalar ve endüstriyel uygulamalar için tercih edildiğinin ana nedenidir.

Bilimsel Python Eko-sistemindeki Yeri

NumPy, tüm bilimsel hesaplama kütüphanelerinin temelidir:

SciPy: İleri düzey bilimsel ve teknik hesaplamalar (optimizasyon, entegrasyon) için NumPy'ı kullanır.

Matplotlib: Grafikler ve görselleştirmeler, NumPy dizilerini girdi olarak kabul eder.

Scikit-learn: Makine öğrenmesi algoritmaları, girdi verilerini (X ve y) ndarray formatında bekler.



Dersin Sonuçları: Veri ve Hesaplama

Yapay Zekâ başarısı, doğru veriye (sayısal, kategorik, sıralı) ve verimli hesaplamaya dayanır.

Veri setleri, düzenli ve temiz olmalıdır (CSV, Tablo yapısı).

NumPy, Python'da yüksek performanslı, çok boyutlu veri manipülasyonu için vazgeçilmez bir araçtır.

ndarray nesnesinin hız ve bellek verimliliği, modern makine öğrenmesi uygulamalarının temelini oluşturur.



VERİ

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

